# Context Reports for the Contract-Violation Handler


| | |
|---|---|
| Document #: | D4301R0 |
| Date: | 2026-10-29 |
| Project: | Programming Language C++ |
| Audience: | EWG, LEWG |


Reply-to: Joshua Berne <jberne4@bloomberg.net>



**Abstract**

When integrating some bug detection facilities with C++26 Contracts there are occasions where violations might be detected and a large amount of additional information could be available if desired. We propose adding a `report` member to `std::contracts::contract_violation` that allows for on-demand generation of such information.


## Contents


**1 Introduction** **2**

**2 Implementation Experience** **3**

**3 Wording Changes** **3**

**4 Conclusion** **5**

## Revision History

Revision 0

- Original version of the paper

# 1 Introduction

When using AddressSanitizer today, a violation that occurs results in a large amount of information being output to `stderr` detailing the nature of the violation, the contents of surrounding memory, and other information that is incredibly useful for diagnosing and fixing the newly discovered bug. AddressSanitizer makes this behavior customizable through a totally separate callback, registered via `__asan_set_error_report_callback`, which can be provided instead of the usual failure callback. To cleanly integrate such behavior with the contract-violation handler we have a number of more nuanced needs:

- Many domains require minimizing the risk and overhead when a violation is detected, so anything that can be done to not generate this information if it isn't going to be needed should be possible.
- Many applications either can't access something output to `stderr` or must not output things to that stream if they are not properly generated by the program itself.

To facilitate these needs, we propose an accessor on `std::contracts::contract_violation` that can trigger the generation of report information on-demand and make it available to the contract-violation handler however it wants.

This accessor, which we call `report`, is a member function of `contract_violation`. This function will return `nullptr` if there is no additional report information available. Otherwise it will return a pointer to a null-terminated string that has implementation-defined contents with more details about the context around the violation.

An important design consideration is whether this member function should be `noexcept`.

- On one hand, this function could conceivably allocate in order to generate the string being produced. It may also fail in other ways.
- On the other hand, the decision to use and support throwing violation handlers is one that should not be undertaken lightly. It would be unfortunate if an allocation failure in generating a report resulted in a program that was expected to terminate immediately kept running in an unexpected and unsupported state.

We therefore recommend that the function be `noexcept`, and any exceptions that occur during report generation should result in producing no report, or a briefer (`const`-storage) message indicating that report generation failed.

In practice the existing sanitizers write reports with known maximum sizes to static storage, and so avoid a risk of failing with an exception.

**Proposal: Add `report` to `contract_violation`**

Add the following member function to `std::contracts::contract_violation` in the header `<contracts>`:

```
const char* report() const noexcept;
```

When invoked, this function will return `nullptr` if no report is available, or a pointer to a null-terminated string that represents context information gathered about the program state where the violation occurred.

Having this facility lets us completely integrate the functionality of the existing sanitizers with the contract-violation handler.

## 2 Implementation Experience

This proposal has been implemented in branches of GCC and Clang, gated on the flag `-fcontracts-p4301` and the feature-test macro `__cpp_lib_contracts_report` (implementation value `202610L`; the wording uses the usual `xxxxxxL` placeholder). Both implementations agree on the `noexcept` decision: `report`'s definition wraps the call to whatever producer populated the report in a `try`/`catch` and returns the fixed string `"Error generating report"` if it throws, so no exception can escape the accessor regardless of what a producer does.

The first producer to use this facility is the AddressSanitizer routing described in D4277R0: on a routed AddressSanitizer violation the sanitizer registers a lazy report populator instead of printing anything itself, and `report` invokes that populator on demand, so the cost of formatting the diagnostic is paid only if a handler asks for it. A runnable example showing `report` returning AddressSanitizer's diagnostic text, built with both GCC and Clang, is available on Compiler Explorer: https://godbolt.org/z/Wcjo8orTK.

## 3 Wording Changes

These wording changes are relative to the C++29 draft sources with git hash d58cd35f, last modified on Thu, 23 Jul 2026 14:08:45 +0200.

### Modified Section Contents


**17 Language support library** **[support]** 4
17.3 Implementation properties [support.limits] 4
17.3.2 Header `<version>` synopsis [version.syn] 4
17.10 Contract-violation handling [support.contract] 4
17.10.1 Header `<contracts>` synopsis [contracts.syn] 4
17.10.3 Class `contract_violation` [support.contract.violation] 5


### Modifications

## 17 Language support library [support]

### 17.3 Implementation properties [support.limits]

#### 17.3.2 Header <version> synopsis [version.syn]

Modify section 17.3.2[version.syn], paragraph 2:

2 Each of the macros defined in <version> is also defined after inclusion of any member of the set of library headers indicated in the corresponding comment in this synopsis.

[*Note 1*: Future revisions of this document might replace the values of these macros with greater values. — *end note*]

```
                                        ⋮ 92 lines elided ⋮
  // <deque>, <queue>, <stack>, <string>
#define __cpp_lib_contracts                     202502L // freestanding, also in <contracts>
#define __cpp_lib_contracts_report              xxxxxxL // freestanding, also in <contracts>↩
#define __cpp_lib_copyable_function             202306L // also in <functional>
#define __cpp_lib_coroutine                     201902L // freestanding, also in <coroutine>

                                        ⋮ 231 lines elided ⋮
```

### 17.10 Contract-violation handling [support.contract]

#### 17.10.1 Header <contracts> synopsis [contracts.syn]

Modify section 17.10.1[contracts.syn], paragraph 1:

1 The header <contracts> defines types for reporting information about contract violations (6.11.2[basic.contract.eval]).

```
// all freestanding
namespace std::contracts {

  enum class assertion_kind : unspecified {
    pre = 1,
    post = 2,
    assert = 3
  };

  enum class evaluation_semantic : unspecified {
    ignore = 1,
    observe = 2,
    enforce = 3,
    quick_enforce = 4
  };

  enum class detection_mode : unspecified {
    predicate_false = 1,
    evaluation_exception = 2
  };

  class contract_violation {
    // no user-accessible constructor
  public:
    contract_violation(const contract_violation&) = delete;
    contract_violation& operator=(const contract_violation&) = delete;
```

```
    see below ~contract_violation();

    const char* comment() const noexcept;
    contracts::detection_mode detection_mode() const noexcept;
    bool is_terminating() const noexcept;
    assertion_kind kind() const noexcept;
    source_location location() const noexcept;
    const char* report() const noexcept;
    evaluation_semantic semantic() const noexcept;
  };

  void invoke_default_contract_violation_handler(const contract_violation&);
}
```

17.10.3 **Class `contract_violation`** **[support.contract.violation]**

Add a new paragraph after 17.10.3[support.contract.violation], paragraph 8:

`const char* report() const noexcept;`

8+a *Returns*: An NTMBS providing additional context about the contract violation if it can be produced; otherwise, `nullptr`. If not null, the lifetime of the storage pointed to by the return value will be usable for as long as `*this` is usable.

8+b *Recommended practice*: Any additional information that might be available about the cause of a contract violation should be produced on-demand when this function is called.

# 4 Conclusion

This small proposal improves the ability to get all of the features made available by the various commonly used sanitizers when they are integrated with implicit contract assertions as introduced by [P3100R8].

# Acknowledgments

Claude (Anthropic) was used for editorial assistance during the preparation of this paper, as well as significant parts of the prototype implementations.

# Bibliography

[P3100R8] Timur Doumler and Joshua Berne, “A framework for systematically addressing undefined behaviour in the C++ Standard”, 2026
http://wg21.link/P3100R8